

8. Codex/Claude에게 실제 작업 지시하는 방식

이제 VS Code에서 매번 이렇게 길게 설명할 필요가 없습니다.

예를 들어 Agent 기능 개발을 맡길 때:

AGEX Agent Framework 기준으로 Agent Resource CRUD와 Publish 기능을 구현해줘.

AGENTS.md와 MASTER.md를 먼저 읽고,

INDEX.md를 통해 필요한 문서를 확인한 뒤 작업해.

Canonical Schema와 현재 구현이 충돌하면 임의로 코드를 기준으로 하지 말고 충돌 내용을 먼저 정리한 뒤 명세 기준으로 수정해.

Workflow라면:

Workflow Engine의 APPROVAL step을 구현해줘.

반드시 AGENTS.md → MASTER.md → INDEX.md 순으로 확인하고

Workflow Engine, Runtime Architecture, Canonical Resource Schema,

IAM/Governance 관련 규칙을 적용해.

WAITING 상태는 Runtime의 core execution state를 사용하고,

approval은 waiting_reason으로 표현해.

이 정도면 됩니다.

9. 가장 중요한 점

“전체 MD를 항상 전부 읽어”라고 하지 않는 게 좋습니다.

오히려 이렇게 해야 합니다.

MASTER

↓

INDEX

↓

현재 작업 관련 문서

↓

Schema

예를 들어 Plugin 작업인데 Cost Estimation, Product Vision, Operations Guide까지 전부 컨텍스트에 넣을 필요는 없습니다.

반대로 중요한 변경, 예를 들어 Agent Permission Model 변경이라면:

MASTER

Agent Framework

IAM

Security

Runtime

Canonical Schema

API Specification

까지 읽혀야 합니다.

즉 문서 수가 많아지는 것이 문제가 아니라 문서 라우팅 체계가 없는 것이 문제입니다.

10. 이 Harness가 만들어내는 효과

이 구조를 제대로 적용하면 Codex/Claude에게 매번 사람이 다음을 설명할 필요가 크게 줄어듭니다.

“Tenant 조심해”

“Model 직접 호출하지 마”

“Agent와 Workflow 다르다”

“Plugin은 Tool이 아니다”

“Published는 수정하면 안 된다”

“DB 직접 보면 안 된다”

이런 내용을 AGENTS.md + MASTER.md가 반복해서 강제하게 됩니다.

그리고 각 Domain의 구체적인 구현은 필요할 때만 해당 명세를 읽게 합니다.

이게 AGEX처럼 방대한 프로젝트에서는 가장 중요합니다.

11. 다음 실제 작업

여기까지 오면 이제 문서 작성 단계가 거의 끝났습니다.

다음 단계는 **Markdown**을 더 작성하는 것이 아니라 실제 **Repository**용 **Harness**를 만드는 작업이 맞습니다.

가장 먼저 만들 것은 다음입니다.

AGENTS.md

CLAUDE.md

MASTER.md

docs/INDEX.md

docs/GLOSSARY.md

그다음:

/specs/schemas/

에 실제 JSON Schema를 생성해야 합니다.

그리고 가장 먼저 Machine-readable Schema로 만들어야 할 순서는:

1. ResourceMetadata
2. ResourceReference
3. TenantContext
4. PrincipalReference
5. ErrorEnvelope
6. EventEnvelope
7. Tenant

8. User / Membership / Role
9. Execution / Task
10. Provider / Model / ModelProfile
11. Agent
12. Workflow
13. Plugin
14. Knowledge
15. Memory